

Instruction Manual: Implementing a 2D Convolution Layer (Conv2d) with Dilation

1 Overview of 2D Convolution

A 2D convolution layer applies learnable filters to an input tensor to produce feature maps, commonly used in convolutional neural networks (CNNs). This manual provides a step-by-step guide to implement a `Conv2d` layer, including forward and backward passes, parameter initialization, and support for dilation (atrous convolution). The implementation uses the `im2col` algorithm for efficiency.

1.1 Key Parameters

- **Input Tensor:** Shape $[B, C_{\text{in}}, H, W]$ (denoted X).
- **Output Tensor:** Shape $[B, C_{\text{out}}, H_{\text{out}}, W_{\text{out}}]$ (denoted Y).
- **Weights (Kernels):** Shape $[C_{\text{out}}, C_{\text{in}}, k_h, k_w]$ (denoted W).
- **Biases (optional):** Shape $[C_{\text{out}}]$ (denoted b).
- **Kernel Size:** (k_h, k_w) , height and width of each filter.
- **Stride:** (s_h, s_w) , step size for sliding the kernel.
- **Padding:** Options: “valid” (no padding), “same” (maintain input size), or custom $(p_{\text{left}}, p_{\text{right}}, p_{\text{top}}, p_{\text{bottom}})$.
- **Padding Mode:** “zero” (zero-padding).
- **Dilation:** (d_h, d_w) , spacing between kernel elements (default $(1, 1)$).
- **Activation Function:** Applied element-wise (e.g., ReLU).
- **Trainable:** Boolean for parameter updates.
- **Bias:** Boolean for including biases.

1.2 Assumptions

- Tensors are 4D with dimensions $[B, C, H, W]$.
- Dilation defaults to $(d_h, d_w) = (1, 1)$ for standard convolution.
- Convolution is implemented using `im2col`.

2 Mathematical Formulation

2.1 Forward Pass

The forward pass computes Y by convolving X with W , adding biases b (if applicable), and applying an activation function σ . For $Y[b, oc, oh, ow]$:

$$Y[b, oc, oh, ow] = \sum_{ic} X[b, ic, ih, iw] \cdot W[oc, ic, kh, kw] + b[oc] \quad (1)$$

Where:

- $ih = oh \cdot s_h + kh \cdot d_h - p_{\text{top}}$
- $iw = ow \cdot s_w + kw \cdot d_w - p_{\text{left}}$

The output is activated:

$$Y[b, oc, oh, ow] = \sigma(Y[b, oc, oh, ow]) \quad (2)$$

2.1.1 Output Dimensions

$$H_{\text{out}} = \left\lfloor \frac{H + p_{\text{top}} + p_{\text{bottom}} - d_h \cdot (k_h - 1) - 1}{s_h} \right\rfloor + 1 \quad (3)$$

$$W_{\text{out}} = \left\lfloor \frac{W + p_{\text{left}} + p_{\text{right}} - d_w \cdot (k_w - 1) - 1}{s_w} \right\rfloor + 1 \quad (4)$$

For “same” padding:

$$p_{h_total} = \max(0, (H_{\text{out}} - 1) \cdot s_h + d_h \cdot (k_h - 1) + 1 - H) \quad (5)$$

$$p_{w_total} = \max(0, (W_{\text{out}} - 1) \cdot s_w + d_w \cdot (k_w - 1) + 1 - W) \quad (6)$$

$$p_{\text{top}} = \left\lfloor \frac{p_{h_total}}{2} \right\rfloor, \quad p_{\text{bottom}} = p_{h_total} - p_{\text{top}} \quad (7)$$

$$p_{\text{left}} = \left\lfloor \frac{p_{w_total}}{2} \right\rfloor, \quad p_{\text{right}} = p_{w_total} - p_{\text{left}} \quad (8)$$

Where $H_{\text{out}} = \lceil H/s_h \rceil$, $W_{\text{out}} = \lceil W/s_w \rceil$.

2.2 Backward Pass

The backward pass computes gradients $\frac{\partial L}{\partial X}$, $\frac{\partial L}{\partial W}$, and $\frac{\partial L}{\partial b}$ given $\frac{\partial L}{\partial Y}$ (denoted **grad_out**).

2.2.1 Gradient w.r.t. Input

$$\frac{\partial L}{\partial X[b, ic, ih, iw]} = \sum_{oc} oc = 0^{C_{\text{out}}-1} \sum_{kh} kh = 0^{k_h-1} \sum_{kw} kw = 0^{k_w-1} \frac{\partial L}{\partial Y'[b, oc, oh, ow]} \cdot W[oc, ic, kh, kw] \quad (9)$$

Where:

- $\frac{\partial L}{\partial Y'} = \sigma'(\text{pre_activation}) \cdot \frac{\partial L}{\partial Y}$
- $oh = \lfloor (ih + p_{\text{top}} - kh \cdot d_h) / s_h \rfloor$
- $ow = \lfloor (iw + p_{\text{left}} - kw \cdot d_w) / s_w \rfloor$

2.2.2 Gradient w.r.t. Weights

$$\frac{\partial L}{\partial W[oc, ic, kh, kw]} = \sum_b b = 0^{B-1} \sum_{oh} oh = 0^{H_{\text{out}}-1} \sum_{ow} ow = 0^{W_{\text{out}}-1} \frac{\partial L}{\partial Y'[b, oc, oh, ow]} \cdot X[b, ic, ih, iw] \quad (10)$$

Where:

- $ih = oh \cdot s_h + kh \cdot d_h - p_{\text{top}}$
- $iw = ow \cdot s_w + kw \cdot d_w - p_{\text{left}}$

2.2.3 Gradient w.r.t. Biases

$$\frac{\partial L}{\partial b[oc]} = \sum_b b = 0^{B-1} \sum_{oh} oh = 0^{H_{\text{out}}-1} \sum_{ow} ow = 0^{W_{\text{out}}-1} \frac{\partial L}{\partial Y'[b, oc, oh, ow]} \quad (11)$$

2.3 Parameter Update

Using an optimizer (e.g., SGD):

$$W \leftarrow W - \eta \cdot \frac{\partial L}{\partial W} \quad (12)$$

$$b \leftarrow b - \eta \cdot \frac{\partial L}{\partial b} \quad (13)$$

Where η is the learning rate.

3 Implementation Details

3.1 Class Structure

The `Conv2d` class includes:

- **Attributes:** Channels, kernel size, stride, padding, dilation, weights (W), biases (b), gradients, input/output caches, activation function.
- **Methods:** Constructor, `forward`, `backward`, `update_parameters`, `reset_grads`, helpers (`pad_input`, `init_output`, `im2col`, `col2im_add`).

3.2 Constructor

3.2.1 Parameter Initialization

Use Xavier initialization:

$$\text{scale} = \sqrt{\frac{6}{\text{fan_in} + \text{fan_out}}} \quad (14)$$

Where $\text{fan_in} = C_{\text{in}} \cdot k_h \cdot k_w$, $\text{fan_out} = C_{\text{out}} \cdot k_h \cdot k_w$.

3.3 Padding

3.4 Output Dimensions

3.5 Im2col Transformation

3.6 Forward Pass

3.7 Col2im Transformation

3.8 Backward Pass

3.9 Parameter Update

3.10 Reset Gradients

4 Example Usage

```
conv = Conv2d(
    in\_channels=3, out\_channels=64, kernel\_size=(3,3),
    \text{stride}=(1,1), padding="same", dilation=(2,2),
    activator=True, bias=True, trainable=True
)
X = random\_tensor(shape=[batch\_size, 3, 28, 28])
Y = conv.forward(X)
grad\_out = random\_tensor(shape=Y.shape)
grad\_input = conv.backward(grad\_out)
optimizer = new SGD()
conv.update\_parameters(optimizer, learning\_rate=0.01)
```

Algorithm 1 Conv2d Constructor

```
1: function CONV2D(in_channels, out_channels, kernel_size=(3,3), stride=(1,1), padding="valid",
   num_padding=(0,0,0,0), padding_mode="zero", dilation=(1,1), activator=null,
   layer_name="Conv2D", trainable=true, bias=true)
2:   if in_channels  $\leq$  0 or out_channels  $\leq$  0 then
3:     throw "Channels must be positive"
4:   end if
5:   if kernel_size[0]  $\leq$  0 or kernel_size[1]  $\leq$  0 then
6:     throw "Kernel size must be positive"
7:   end if
8:   if stride[0]  $\leq$  0 or stride[1]  $\leq$  0 then
9:     throw "Stride must be positive"
10:  end if
11:  if dilation[0]  $\leq$  0 or dilation[1]  $\leq$  0 then
12:    throw "Dilation must be positive"
13:  end if
14:  if padding  $\notin$  ["valid", "same", "none"] then
15:    throw "Invalid padding mode"
16:  end if
17:  if padding_mode  $\neq$  "zero" then
18:    throw "Only zero-padding is supported"
19:  end if
20:  if num_padding[0,1,2,3]  $<$  0 then
21:    throw "Padding values must be non-negative"
22:  end if
23:  this.in_channels  $\leftarrow$  in_channels
24:  this.out_channels  $\leftarrow$  out_channels
25:  this.kernel_size  $\leftarrow$  kernel_size
26:  this.stride  $\leftarrow$  stride
27:  this.padding  $\leftarrow$  padding
28:  this.num_padding  $\leftarrow$  num_padding
29:  this.padding_mode  $\leftarrow$  padding_mode
30:  this.dilation  $\leftarrow$  dilation
31:  this.layer_name  $\leftarrow$  layer_name or "Conv2D_{in_channels}x{out_channels}"
32:  this.trainable  $\leftarrow$  trainable
33:  this.bias  $\leftarrow$  bias
34:  if activator is null then
35:    this.activator  $\leftarrow$  new ReLU()
36:  else
37:    this.activator  $\leftarrow$  activator
38:  end if
39:  init_params_and_grads()
40: end function
```

Algorithm 2 Initialize Parameters and Gradients

```
1: function INIT_PARAMS_AND_GRADS
2:   fan_in  $\leftarrow$  in_channels  $\cdot$  kernel_size[0]  $\cdot$  kernel_size[1]
3:   fan_out  $\leftarrow$  out_channels  $\cdot$  kernel_size[0]  $\cdot$  kernel_size[1]
4:   scale  $\leftarrow \sqrt{6/(\text{fan\_in} + \text{fan\_out})}$ 
5:   weights  $\leftarrow$  random_uniform(-scale, scale, shape=[out_channels, in_channels, kernel_size[0], kernel_size[1]])
6:   grad_weights  $\leftarrow$  zeros(shape=[out_channels, in_channels, kernel_size[0], kernel_size[1]])
7:   if bias then
8:     biases  $\leftarrow$  zeros(shape=[out_channels])
9:     grad_biases  $\leftarrow$  zeros(shape=[out_channels])
10:  else
11:    biases  $\leftarrow$  empty_tensor()
12:    grad_biases  $\leftarrow$  empty_tensor()
13:  end if
14: end function
```

Algorithm 3 Pad Input

```
1: function PAD_INPUT(X)
2:   p_left, p_right, p_top, p_bottom  $\leftarrow$  num_padding
3:   if padding = "same" then
4:     k_h, k_w  $\leftarrow$  kernel_size
5:     s_h, s_w  $\leftarrow$  stride
6:     d_h, d_w  $\leftarrow$  dilation
7:     H, W  $\leftarrow$  X.shape[2], X.shape[3]
8:     out_h  $\leftarrow \lceil H/s_h \rceil$ 
9:     out_w  $\leftarrow \lceil W/s_w \rceil$ 
10:    p_h_total  $\leftarrow \max(0, (\text{out\_h} - 1) \cdot s_h + d_h \cdot (k_h - 1) + 1 - H)$ 
11:    p_w_total  $\leftarrow \max(0, (\text{out\_w} - 1) \cdot s_w + d_w \cdot (k_w - 1) + 1 - W)$ 
12:    p_top  $\leftarrow \lfloor p\_h\_total/2 \rfloor$ 
13:    p_bottom  $\leftarrow p\_h\_total - p\_top$ 
14:    p_left  $\leftarrow \lfloor p\_w\_total/2 \rfloor$ 
15:    p_right  $\leftarrow p\_w\_total - p\_left$ 
16:  end if
17:  if p_left = 0 and p_right = 0 and p_top = 0 and p_bottom = 0 then
18:    return X
19:  end if
20:  padded_X  $\leftarrow$  zeros(shape=[X.shape[0], X.shape[1], H + p_top + p_bottom, W + p_left + p_right])
21:  padded_X[:, :, p_top : H + p_top, p_left : W + p_left]  $\leftarrow$  X
22:  return padded_X
23: end function
```

Algorithm 4 Initialize Output

```
1: function INIT_OUTPUT(X)
2:    $B, C, H, W \leftarrow X.\text{shape}$ 
3:    $k_h, k_w \leftarrow \text{kernel\_size}$ 
4:    $s_h, s_w \leftarrow \text{stride}$ 
5:    $d_h, d_w \leftarrow \text{dilation}$ 
6:    $p_{\text{top}}, p_{\text{bottom}}, p_{\text{left}}, p_{\text{right}} \leftarrow \text{num\_padding}$ 
7:   if padding in ["valid", "none"] then
8:      $\text{out}_h \leftarrow \lfloor (H + p_{\text{top}} + p_{\text{bottom}} - d_h \cdot (k_h - 1) - 1) / s_h \rfloor + 1$ 
9:      $\text{out}_w \leftarrow \lfloor (W + p_{\text{left}} + p_{\text{right}} - d_w \cdot (k_w - 1) - 1) / s_w \rfloor + 1$ 
10:    if  $(H + p_{\text{top}} + p_{\text{bottom}} - d_h \cdot (k_h - 1) - 1) \bmod s_h \neq 0$  or  $(W + p_{\text{left}} + p_{\text{right}} - d_w \cdot$ 
     $(k_w - 1) - 1) \bmod s_w \neq 0$  then
11:      throw "Non-integer output dimensions"
12:    end if
13:  else if padding = "same" then
14:     $\text{out}_h \leftarrow \lceil H / s_h \rceil$ 
15:     $\text{out}_w \leftarrow \lceil W / s_w \rceil$ 
16:  end if
17:  if  $\text{out}_h \leq 0$  or  $\text{out}_w \leq 0$  then
18:    throw "Invalid output dimensions"
19:  end if
20:  output  $\leftarrow \text{zeros}(\text{shape}=[B, \text{out\_channels}, \text{out}_h, \text{out}_w])$ 
21:  return out_h, out_w
22: end function
```

5 Notes on Dilation

- Dilation expands the receptive field, spacing kernel elements by d_h, d_w .
- $ih = oh \cdot s_h + kh \cdot d_h$, $iw = ow \cdot s_w + kw \cdot d_w$ in `im2col`.
- Effective kernel size: $d_h \cdot (k_h - 1) + 1$, $d_w \cdot (k_w - 1) + 1$.

6 Error Handling

- Validate input dimensions and parameters.
- Check for non-integer or negative output dimensions.
- Ensure gradient shape matches.

7 Optimization Tips

- Use efficient matrix libraries (e.g., BLAS, Eigen, NumPy).
- Cache intermediate results.
- Parallelize loops in `im2col` and `col2im_add`.
- Consider GPUs for CUDA.

Algorithm 5 Im2col Transformation

```
1: function IM2COL(input)
2:    $B, C, H, W \leftarrow \text{input.shape}$ 
3:    $k_h, k_w \leftarrow \text{kernel\_size}$ 
4:    $s_h, s_w \leftarrow \text{stride}$ 
5:    $d_h, d_w \leftarrow \text{dilation}$ 
6:    $\text{out\_h} \leftarrow \lfloor (H - d_h \cdot (k_h - 1) - 1) / s_h \rfloor + 1$ 
7:    $\text{out\_w} \leftarrow \lfloor (W - d_w \cdot (k_w - 1) - 1) / s_w \rfloor + 1$ 
8:   if  $\text{out\_h} \leq 0$  or  $\text{out\_w} \leq 0$  then
9:     throw "Invalid output dimensions in im2col"
10:  end if
11:   $\text{col\_height} \leftarrow k_h \cdot k_w \cdot \text{in\_channels}$ 
12:   $\text{col\_width} \leftarrow B \cdot \text{out\_h} \cdot \text{out\_w}$ 
13:   $\text{col\_matrix} \leftarrow \text{zeros}(\text{shape}=[\text{col\_height}, \text{col\_width}])$ 
14:   $\text{col\_idx} \leftarrow 0$ 
15:  for  $b$  in 0 to  $B - 1$  do
16:    for  $oh$  in 0 to  $\text{out\_h} - 1$  do
17:      for  $ow$  in 0 to  $\text{out\_w} - 1$  do
18:         $\text{row\_idx} \leftarrow 0$ 
19:        for  $ic$  in 0 to  $\text{in\_channels} - 1$  do
20:          for  $kh$  in 0 to  $k_h - 1$  do
21:            for  $k_w$  in 0 to  $k_w - 1$  do
22:               $ih \leftarrow oh \cdot s_h + kh \cdot d_h$ 
23:               $iw \leftarrow ow \cdot s_w + kw \cdot d_w$ 
24:              if  $ih < H$  and  $iw < W$  then
25:                 $\text{col\_matrix}[\text{row\_idx}, \text{col\_idx}] \leftarrow \text{input}[b, ic, ih, iw]$ 
26:              end if
27:               $\text{row\_idx} \leftarrow \text{row\_idx} + 1$ 
28:            end for
29:          end for
30:        end for
31:         $\text{col\_idx} \leftarrow \text{col\_idx} + 1$ 
32:      end for
33:    end for
34:  end for
35:  return  $\text{col\_matrix}$ 
36: end function
```

Algorithm 6 Forward Pass

```
1: function FORWARD(X)
2:   in_cache  $\leftarrow$  X
3:    $B, C, H, W \leftarrow$  X.shape
4:   if  $C \neq$  in_channels then
5:     throw "Input channels mismatch"
6:   end if
7:   if  $H <$  kernel_size[0] or  $W <$  kernel_size[1] then
8:     throw "Input dimensions too small"
9:   end if
10:  input_padded  $\leftarrow$  pad_input(X)
11:  out_h, out_w  $\leftarrow$  init_output(X)
12:  col_matrix  $\leftarrow$  im2col(input_padded)
13:  weight_matrix  $\leftarrow$  weights.reshape([out_channels,  $k_h \cdot k_w$ , in_channels])
14:  result  $\leftarrow$  weight_matrix  $\cdot$  col_matrix
15:  if bias then
16:    bias_broadcasted  $\leftarrow$  biases.reshape([out_channels, 1]).broadcast([1,  $B \cdot$  out_h  $\cdot$  out_w])
17:    result  $\leftarrow$  result + bias_broadcasted
18:  end if
19:  pre_activation  $\leftarrow$  result.reshape([B, out_channels, out_h, out_w])
20:  output  $\leftarrow$  activator.forward(pre_activation)
21:  return output
22: end function
```

Algorithm 7 Col2im Add

```
1: function COL2IM_ADD(col_matrix, grad_input)
2:    $B, C, H, W \leftarrow$  grad_input.shape
3:    $k_h, k_w \leftarrow$  kernel_size
4:    $s_h, s_w \leftarrow$  stride
5:    $d_h, d_w \leftarrow$  dilation
6:   col_idx  $\leftarrow$  0
7:   for b in 0 to  $B - 1$  do
8:     for oh in 0 to out_h-1 do
9:       for ow in 0 to out_w-1 do
10:        row_idx  $\leftarrow$  0
11:        for ic in 0 to in_channels-1 do
12:          for kh in 0 to  $k_h - 1$  do
13:            for kw in 0 to  $k_w - 1$  do
14:              ih  $\leftarrow$  oh  $\cdot s_h + kh \cdot d_h$ 
15:              iw  $\leftarrow$  ow  $\cdot s_w + kw \cdot d_w$ 
16:              if ih  $< H$  and iw  $< W$  then
17:                grad_input[b, ic, ih, iw]  $\leftarrow$  grad_input[b, ic, ih, iw] + col_matrix[row_idx,
col_idx]
18:              end if
19:              row_idx  $\leftarrow$  row_idx + 1
20:            end for
21:          end for
22:        end for
23:        col_idx  $\leftarrow$  col_idx + 1
24:      end for
25:    end for
26:  end for
27: end function
```

Algorithm 8 Backward Pass

```
1: function BACKWARD(grad_out)
2:   if grad_out.shape  $\neq$  [B, out_channels, out_h, out_w] then
3:     throw "Gradient output dimensions mismatch"
4:   end if
5:   grad_input  $\leftarrow$  zeros(shape=in_cache.shape)
6:   if trainable then
7:     grad_weights  $\leftarrow$  zeros(shape=weights.shape)
8:     if bias then
9:       grad_biases  $\leftarrow$  zeros(shape=biases.shape)
10:    end if
11:  end if
12:  input_padded  $\leftarrow$  pad_input(in_cache)
13:  grad_pre_activation  $\leftarrow$  activator.backward(grad_out)
14:  grad_out_matrix  $\leftarrow$  grad_pre_activation.reshape([out_channels, B  $\cdot$  out_h  $\cdot$  out_w])
15:  if trainable then
16:    col_matrix  $\leftarrow$  im2col(input_padded)
17:    weight_grad_matrix  $\leftarrow$  grad_out_matrix  $\cdot$  col_matrix.transpose()
18:    grad_weights  $\leftarrow$  weight_grad_matrix.data().reshape([out_channels, in_channels, k_h, k_w])
19:    if bias then
20:      grad_biases  $\leftarrow$  grad_out_matrix.sum(axis=1)
21:    end if
22:  end if
23:  weight_matrix_weights.reshape([k_h  $\cdot$  k_w  $\cdot$  in_channels, grad_out_matrix, out_channels])
24:  grad_input_col  $\leftarrow$  grad_out_matrix * weight_matrix_weights.T
25:  col2im_add(grad_input_col, grad_input)
26:  return grad_input
27: end function
```

Algorithm 9 Update Parameters

```
1: function UPDATE_PARAMETERS(optimizer, learning_rate)
2:   if not trainable then
3:     return
4:   end if
5:   if weights.shape  $\neq$  grad_weights.shape then
6:     throw "Weight and gradient dimension mismatch"
7:   end if
8:   if bias and biases.shape  $\neq$  grad_biases.shape then
9:     throw "Bias and gradient dimension mismatch"
10:  end if
11:  optimizer.update(this, learning_rate)
12: end function
```

Algorithm 10 Reset Gradients

```
1: function RESET_GRADS
2:   if trainable then
3:     grad_weights  $\leftarrow$  zeros(shape=grad_weights.shape)
4:     if bias then
5:       grad_biases  $\leftarrow$  zeros(shape=grad_biases.shape)
6:     end if
7:   end if
8: end function
```
